

TeamOps Microservices Architecture

*Study guide and migration reference
September 2026*

1. Purpose

This document is a practical architecture reference for evolving TeamOps from a Django monolith into a set of independently deployable microservices. The goal is not to maximize the number of services. The goal is to make business boundaries, ownership, and scaling clear while preserving delivery speed and reliability.

2. Target architecture

Clients call an API gateway or Backend for Frontend (BFF). The gateway routes requests to the service that owns the capability. Each service owns its own data and can be deployed independently.

```
Frontend
|-- API gateway / BFF
|  |-- Auth service      -> PostgreSQL database
|  |-- Product / Workspace -> PostgreSQL database
|  |-- Project service   -> PostgreSQL database
|  |-- Task service      -> PostgreSQL database
|  '-- Notification service -> database and message queue
```

Shared infrastructure includes a message broker such as RabbitMQ, Redis where justified, centralized logs, metrics, tracing, and secret management.

3. Recommended service boundaries

Auth / Identity	Users, registration, passwords, login, refresh tokens, email verification, password reset, roles, and authentication audit records.
Product / Workspace	Organizations, workspaces, workspace membership, invitation lifecycle, and subscription-related ownership.
Project	Projects, project configuration, and project membership policies.
Task	Tasks, comments, labels, assignees, due dates, and workflow/status changes.
Notification	Email, in-app notifications, delivery preferences, and fan-out of activity events.

4. Non-negotiable design rules

1. Every service owns its database. A service must never read or write another service's tables.
2. Share contracts, not domain models. A small versioned package may contain logging helpers, JWT validation utilities, standardized error formats, and event schemas. It must not contain shared ORM models.
3. Use synchronous HTTP/gRPC calls only when an immediate response is required. Use events for propagation and background work.
4. Design every consumer to be idempotent: processing the same event twice must be safe.
5. Do not use distributed database transactions. Prefer local transactions, retries, compensating actions, and the transactional outbox pattern.

5. Authentication and authorization model

The Auth service is the identity authority. It authenticates a user and issues a short-lived access token plus a longer-lived refresh token. Prefer asymmetric JWT signing (for example RS256 or EdDSA). The Auth service keeps the private key; other services retrieve public keys from a JWKS endpoint and validate tokens locally.

An access token should minimally include the subject (the user UUID), issued-at and expiry times, issuer, audience, and only stable authorization claims that are appropriate for the token lifetime. Do not place sensitive profile data or large membership lists in a JWT.

The Product, Project, and Task services should validate the JWT signature locally rather than call Auth for every request. They then authorize locally based on the resource: for example, the Task service checks whether the authenticated user UUID is permitted to access that task's workspace and project.

Use UUIDs as externally visible IDs. They avoid exposing sequential record counts and work well across service boundaries.

6. Communication patterns

Request-response	Use an API call when the caller needs an answer now: creating a task, reading a project, or validating a command.
Domain event	Use RabbitMQ initially for events such as user.created, workspace.member_added, project.created, task.assigned, and task.completed. It is an approachable first broker for this system.
Outbox pattern	When a service changes its database, it writes the domain event to an outbox table in the same local transaction. A worker publishes that outbox record to the broker. This prevents a successful database commit from silently losing its event.

7. Migration plan

Phase 1: strengthen the modular monolith	Keep the current Django deployment while the product is small. Define module boundaries, APIs, ownership, automated tests, and Docker-based local development.
Phase 2: extract Auth	Turn the current accounts app into its own Auth service. Add UUID user IDs, login, access and refresh token rotation, password reset, email verification, throttling, security/audit logs, health checks, migrations, and its own PostgreSQL database.
Phase 3: introduce Product / Workspace	Create a separate service and database for workspaces and memberships. The workspace service stores the auth user UUID as an external reference, never as a foreign key to the Auth database.
Phase 4: extract Project and Task	Extract only after those domains have real functionality and clear ownership. Start with a single Task service rather than splitting comments, labels, and assignments into separate services.
Phase 5: introduce events	Add RabbitMQ and the outbox pattern for cross-service activity. Use events for notifications, search indexing, analytics, and projections.
Phase 6: production readiness	Add structured logs with correlation IDs, metrics, tracing, backups, alerting, CI/CD, image scanning, secret management, and rate limits. Kubernetes is optional; Docker Compose is sufficient for local development and early deployment.

8. Repository layout

TeamOps/
services/

```
auth-service/  
product-service/  
project-service/  
task-service/  
notification-service/  
packages/  
  common/          # Small shared technical utilities only  
  contracts/       # Versioned API and event schemas  
infrastructure/  
  docker-compose.yml  
gateway/
```

9. Operational checklist

Security	TLS everywhere, hashed passwords, refresh token rotation and revocation, key rotation, request throttling, strict CORS policy, secrets outside source control, dependency scanning, and audit logs for authentication and privileged actions.
Reliability	Health and readiness endpoints, timeouts, retry limits, circuit breakers where appropriate, idempotency keys for write APIs, backups that are tested by restoration, and dead-letter queues for failed events.
Observability	Structured JSON logs, a correlation/trace ID propagated between gateway and services, metrics for latency/error rates/queue depth, and distributed tracing using OpenTelemetry.
Data	Per-service migrations, no cross-database joins, explicit data retention policy, and event schemas that are backward compatible.

10. What to avoid

1. Do not split into many services before the domain warrants it. Each service adds deployment, testing, monitoring, security, and incident-response cost.
2. Do not use a shared database as the integration mechanism.
3. Do not make Auth a required network call for each user request.
4. Do not add Kubernetes merely because services exist. Use it when operational needs justify it.
5. Do not make cross-service calls in long chains. They are slow and fragile; prefer a direct owner or asynchronous events.

11. First implementation milestone

The best first milestone is an independently deployable Auth service. It should provide registration, login, token refresh, logout/revocation, email verification, password reset, JWKS, OpenAPI documentation, health endpoints, automated tests, and its own PostgreSQL database. Then create the Product / Workspace service and integrate it by validating Auth-issued JWTs locally.

This sequence gives TeamOps a secure identity foundation without prematurely creating a complex distributed system.